การประมวลผลแบบกลุ่มเมฆ

CLOUD COMPUTING

IT368

Lecture 5

Cloud Architecture Design

การออกแบบสถาปัตยกรรมคลาวด์ที่เหมาะสม
เพื่อรองรับงานในโลกปัจจุบันและอนาคต



Scalable Systems | Efficient Design | Optimized Cost | Reliable Future

การออกแบบ Cloud คืออะไร?

การออกแบบ Cloud (Cloud Architecture Design) คือ กระบวนการวางแผน
และออกแบบโครงสร้างของระบบสารสนเทศบนคลาวด์ ให้สามารถตอบสนอง
ความต้องการของธุรกิจได้อย่างมีประสิทธิภาพ ปลอดภัย ยืดหยุ่น และคุ้มค่าต่อการใช้งาน



เป้าหมายของการออกแบบ Cloud

"ออกแบบสถาปัตยกรรมและองค์ประกอบต่าง ๆ บนคลาวด์
ให้เหมาะสมกับความต้องการขององค์กร ทั้งในปัจจุบันและอนาคต"

ครอบคลุมการออกแบบในหลายมิติ เช่น

- โครงสร้างสถาปัตยกรรมระบบ
- ทรัพยากรและขนาดการใช้งาน
- ความปลอดภัย
- เครือข่ายและการเชื่อมต่อ
- ค่าใช้จ่ายและการเพิ่มประสิทธิภาพ
- การรองรับการขยายตัวในอนาคต



**"การออกแบบ Cloud ที่ตี ช่วยให้องค์กรใช้เทคโนโลยีได้เต็มศักยภาพ
พร้อมรับการเปลี่ยนแปลงในอนาคต"**

ทำไมจึงมีการออกแบบ Cloud?

แม้ว่า Cloud Provider จะช่วยให้เราสร้างทรัพยากรได้ง่ายและรวดเร็ว
แต่การมีทรัพยากรบน Cloud ไม่ได้หมายความว่าจะได้ระบบที่ดีโดยอัตโนมัติ
การออกแบบ Cloud จึงมีความสำคัญ เพื่อให้ระบบตรงกับความต้องการของธุรกิจ
ใช้งานได้อย่างมีประสิทธิภาพ ปลอดภัย และคุ้มค่าต่อการลงทุน



หากไม่มีการออกแบบที่ดี อาจเกิดปัญหา เช่น

สิ้นเปลืองค่าใช้จ่าย
เลือกทรัพยากรใหญ่เกินความจำเป็น จ่ายค่าใช้จ่ายที่ไม่ได้ใช้งาน

ประสิทธิภาพไม่เพียงพอ
ทรัพยากรเล็กเกินไป ทำให้ระบบช้า หรือไม่สามารถรองรับผู้ใช้ได้

มีจุดล้มเหลวเพียงจุดเดียว (Single Point of Failure)
หากเครื่องเดียวเสีย ระบบทั้งหมดอาจหยุดให้บริการ

ความเสี่ยงด้านความปลอดภัย
ไม่มีการวางมาตรการป้องกันที่เหมาะสม

ขยายระบบได้ยาก
เมื่อผู้ใช้เพิ่มขึ้น อาจต้องปรับเปลี่ยนโครงสร้างครั้งใหญ่

การออกแบบ Cloud ช่วยให้เรา...

ประโยชน์ของการออกแบบ Cloud

ประสิทธิภาพที่เหมาะสม
ระบบตอบสนองได้ตามความต้องการ (Performance)

รองรับการเติบโต
สามารถขยายระบบได้ตามปริมาณงาน (Scalability)

ความพร้อมใช้งานสูง
ยังให้บริการได้แม้บางส่วนเกิดปัญหา (Availability)

ปลอดภัยและเป็นไปตามข้อกำหนด
ปกป้องระบบและข้อมูล (Security)

คุ้มค่าการลงทุน
ใช้ทรัพยากรเท่าที่จำเป็น และสามารถประมาณค่าใช้จ่ายได้ (Cost Efficiency)

"การออกแบบ Cloud คือการหา **จุดสมดุลที่ดีที่สุด ระหว่างประสิทธิภาพ การขยายตัว**
ความพร้อมใช้งาน ความปลอดภัย และค่าใช้จ่าย เพื่อให้ระบบสนับสนุนเป้าหมายของธุรกิจได้อย่างยั่งยืน"

กระบวนการในการออกแบบ Cloud

การออกแบบ Cloud เป็นกระบวนการแบบเป็นขั้นตอน ตั้งแต่การเข้าใจความต้องการของระบบ ไปจนถึงการปรับปรุงให้เหมาะสม เพื่อให้ได้สถาปัตยกรรมที่ตอบโจทย์ทั้งด้านเทคนิคและธุรกิจ

From Requirements to Value

Design
Build
Optimize
Repeat

1 Requirement Analysis



- เข้าใจเป้าหมายของธุรกิจ
- เก็บความต้องการ ทั้ง Functional และ Non-Functional
- กำหนดข้อจำกัด และงบประมาณเบื้องต้น

“ต้องการอะไร?”

2 Workload Analysis



- วิเคราะห์ลักษณะงาน (Workload)
- ประเมินปริมาณผู้ใช้ และรูปแบบการใช้งาน
- ระบุความต้องการด้าน Compute, Storage, Network, Database

“งานมีลักษณะอย่างไร?”

3 Architecture Design



- ออกแบบสถาปัตยกรรม และเลือกบริการ Cloud
- กำหนด Component และความสัมพันธ์
- พิจารณารูปแบบ Single, Multi-Tier หรือ High Availability

“ออกแบบอย่างไร?”

4 Sizing



- คำนวณขนาดทรัพยากร ที่ต้องใช้
- CPU, Memory, Storage, Network
- เผื่อการเติบโต และกำหนด Capacity Headroom

“ต้องใช้ทรัพยากร เท่าไร?”

5 Security Design



- ออกแบบการป้องกัน ในทุกระดับ (Defense in Depth)
- กำหนดการเข้าถึง (IAM, Least Privilege)
- วางแผนการเข้ารหัส การสำรองข้อมูล และการตรวจสอบ

“จะปกป้องระบบอย่างไร?”

6 Cost Estimation



- ประมาณค่าใช้จ่าย ของแต่ละบริการ
- รวมค่า Compute, Storage, Database, Network และอื่น ๆ
- เปรียบเทียบกับ งบประมาณที่กำหนด

“มีค่าใช้จ่ายเท่าไร?”

7 Optimization



- ตรวจสอบประสิทธิภาพ และค่าใช้จ่าย
- ปรับขนาดทรัพยากร (Right Sizing)
- ใช้ Auto Scaling และเลือก Pricing Model ที่เหมาะสม
- ปรับปรุงอย่างต่อเนื่อง

“ปรับปรุงให้ดีขึ้นได้อย่างไร?”

กระบวนการนี้ทำซ้ำได้เสมอ

เพราะความต้องการของธุรกิจและปริมาณงานอาจเปลี่ยนแปลงในอนาคต

Requirement Analysis

การวิเคราะห์ความต้องการของระบบ

Requirement Analysis คือขั้นตอนแรกในการออกแบบ Cloud เพื่อทำความเข้าใจว่า ระบบต้องทำอะไร มีข้อกำหนดอะไรบ้าง และมีเป้าหมายอย่างไร ทั้งในด้านการทำงานของระบบ และด้านคุณภาพ เพื่อใช้เป็นพื้นฐานในการออกแบบสถาปัตยกรรม เลือกบริการ และกำหนดขนาดทรัพยากร



Functional Requirements

ความสามารถที่ระบบต้องมี (ทำอะไรได้บ้าง)

- **การจัดการผู้ใช้** — สมัครสมาชิก เข้าสู่ระบบ
- **การทำธุรกรรม** — สั่งซื้อสินค้า ชำระเงิน
- **การจัดการข้อมูล** — จัดเก็บ ค้นหา และแสดงข้อมูล
- **ฟังก์ชันเฉพาะ** — เช่น การแจ้งเตือน การออกรายงาน

Non-Functional Requirements

ข้อกำหนดด้านคุณภาพของระบบ (ต้องดีแค่ไหน)

- **Performance** — เช่น Response Time < 2 วินาที
- **Availability** — เช่น 99.9% (ระบบพร้อมใช้งาน)
- **Scalability** — รองรับผู้ใช้ที่เพิ่มขึ้นได้
- **Security** — เช่น เข้ารหัสข้อมูล ควบคุมการเข้าถึง
- **Data & Backup** — เช่น สำรองข้อมูลทุกวัน กู้คืนได้ภายใน 1 ชั่วโมง
- **Cost** — เช่น ไม่เกิน 20,000 บาท/เดือน

ขั้นตอนการทำ Requirement Analysis

1. **ศึกษาความต้องการจากผู้มีส่วนเกี่ยวข้อง** (Stakeholders)
2. **รวบรวมและจัดกลุ่มความต้องการ** (Functional / Non-Functional)
3. **ตรวจสอบและจัดลำดับความสำคัญ** (Prioritize)
4. **สรุปเป็นเอกสาร Requirement** (เช่น Requirement Specification)

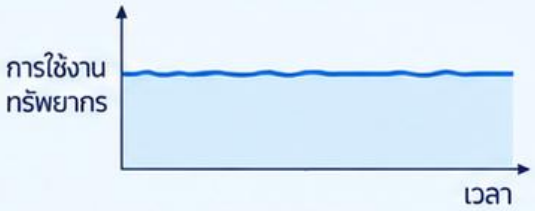
> “Requirement **ที่ชัดเจน** คือจุดเริ่มต้นของการออกแบบ Cloud **ที่เหมาะสม** **ช่วยลดความเสี่ยง ลดค่าใช้จ่าย และทำให้ระบบตอบโจทย์ธุรกิจได้จริง**”

ประเภทของ Workload

Workload คือ ลักษณะของงานที่ระบบต้องประมวลผล ซึ่งสามารถแบ่งได้หลายรูปแบบ การเข้าใจประเภทของ Workload ช่วยให้เราเลือกบริการและออกแบบสถาปัตยกรรมบน Cloud ได้เหมาะสม ทั้งในด้านประสิทธิภาพ การขยายตัว และการควบคุมค่าใช้จ่าย

1 Static Workload
โหลดคงที่

มีการใช้งานทรัพยากรค่อนข้างสม่ำเสมอ ไม่เปลี่ยนแปลงมากในช่วงเวลา



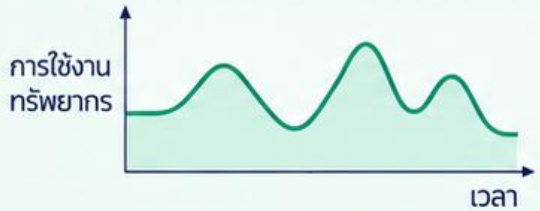
ตัวอย่างงาน

- ระบบงานภายในองค์กร (Internal System)
- ERP / HR / Finance System
- Database ที่มีการใช้งานสม่ำเสมอ
- Web Application ที่มีผู้ใช้คงที่

เหมาะกับการใช้ทรัพยากรแบบคงที่ หรือใช้ Reserved Instance เพื่อลดค่าใช้จ่าย

2 Variable Workload
โหลดแปรผัน

มีการใช้งานทรัพยากรขึ้นลงตามช่วงเวลา แต่ไม่มีรูปแบบที่แน่นอนตายตัว



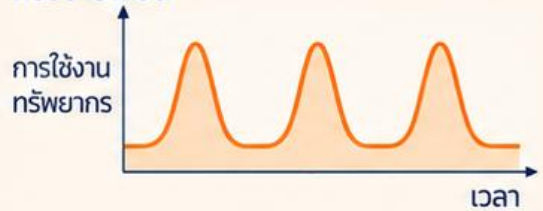
ตัวอย่างงาน

- E-Commerce (ยอดผู้ใช้เปลี่ยนตามวัน)
- Web Application ทั่วไป
- Streaming Service (จำนวนผู้ชมขึ้นลง)
- Application ที่มีผู้ใช้มากในช่วงเวลาทำงาน

เหมาะกับการใช้ Auto Scaling เพื่อปรับขนาดทรัพยากรตามการใช้งาน

3 Periodic Workload
โหลดตามรอบเวลา

มีการใช้งานทรัพยากรสูงเป็นช่วง ๆ ตามรอบเวลาที่แน่นอน เช่น รายวัน รายสัปดาห์ หรือรายเดือน



ตัวอย่างงาน

- รายงานประจำวัน / ประจำเดือน
- Batch Processing
- Data Backup
- ETL (Extract, Transform, Load)
- ระบบประมวลผลสิ้นเดือน / สิ้นไตรมาส

เหมาะกับการตั้งเวลาเพิ่ม/ลดทรัพยากรตามรอบงาน (Scheduled Scaling)

4 Unpredictable Workload
โหลดไม่แน่นอน

มีการใช้งานทรัพยากรที่คาดการณ์ได้ยาก อาจเพิ่มขึ้นอย่างฉับพลันและไม่สม่ำเสมอ



ตัวอย่างงาน

- ระบบที่ถูกกระแส (Viral Application)
- Flash Sale
- News & Media (เหตุการณ์ฉุกเฉิน)
- Online Event / Live Streaming
- Start-up ที่ยังไม่ทราบรูปแบบผู้ใช้ชัดเจน

เหมาะกับการออกแบบให้ยืดหยุ่นสูง ใช้ Auto Scaling และบริการแบบ Serverless ได้

“การเข้าใจประเภทของ Workload ช่วยให้เลือกบริการ Cloud ได้เหมาะสม รองรับการใช้งานได้อย่างมีประสิทธิภาพ และควบคุมค่าใช้จ่ายได้ดียิ่งขึ้น”

สิ่งที่ผู้ให้บริการ Cloud เตรียมให้

ผู้ให้บริการ Cloud (เช่น AWS, Microsoft Azure, Google Cloud) มีบริการพื้นฐานที่ครอบคลุม ทรัพยากร โครงสร้างพื้นฐาน และเครื่องมือสำเร็จรูป เพื่อให้เราสามารถสร้างและให้บริการระบบได้อย่างรวดเร็ว โดยไม่ต้องลงทุนและดูแลฮาร์ดแวร์เอง

aws | Microsoft Azure | Google Cloud



Compute
ทรัพยากรสำหรับประมวลผล

ให้บริการเครื่องเสมือน เซิร์ฟเวอร์ คอนเทนเนอร์ และบริการแบบไร้เซิร์ฟเวอร์ เพื่อรันแอปพลิเคชัน

ตัวอย่างบริการ

- Virtual Machine (VM)
- Container Service (เช่น ECS, AKS, GKE)
- Serverless (เช่น AWS Lambda)



Storage
บริการจัดเก็บข้อมูล

ให้บริการพื้นที่จัดเก็บข้อมูลที่ปลอดภัย เชื่อถือได้ และสามารถขยายได้ตามต้องการ

ตัวอย่างบริการ

- Object Storage (เช่น S3, Blob, GCS)
- Block Storage (เช่น EBS, Disk)
- File Storage (เช่น EFS, Azure Files)



Database
บริการฐานข้อมูล

ให้บริการฐานข้อมูลหลากหลายประเภท ทั้งแบบจัดการเต็มรูปแบบ (Managed) เพื่อลดภาระการดูแลรักษา

ตัวอย่างบริการ

- Relational Database (เช่น RDS, Azure SQL, Cloud SQL)
- NoSQL Database (เช่น DynamoDB, Cosmos DB)
- In-Memory Database (เช่น ElastiCache, Memorystore)



Networking
บริการเครือข่าย

ให้บริการโครงข่ายที่ปลอดภัย ยืดหยุ่น และเชื่อมต่อกับอินเทอร์เน็ตและระบบอื่น ๆ ได้อย่างมีประสิทธิภาพ

ตัวอย่างบริการ

- Virtual Private Cloud (VPC)
- Load Balancer (เช่น ALB, NLB)
- DNS (เช่น Route 53, Cloud DNS)
- CDN (เช่น CloudFront, Cloud CDN)



"ผู้ให้บริการ Cloud เตรียมโครงสร้างพื้นฐานและบริการที่จำเป็นไว้ให้
เพื่อให้เราสามารถมุ่งเน้นที่การพัฒนาแอปพลิเคชันและสร้างคุณค่าให้กับธุรกิจได้รวดเร็วยิ่งขึ้น"

เลือก Computing แบบไหน เหมาะกับ Workload?

แต่ละรูปแบบมีจุดเด่นที่แตกต่างกัน เลือกให้เหมาะกับลักษณะของงาน เป้าหมาย และข้อจำกัดของคุณ



Virtual Machine (VM)

เครื่องเสมือนแบบเต็มรูปแบบ

เหมาะกับ

- งานที่ต้องการควบคุมสูง
- ระบบเดิม (Legacy)
- แอปพลิเคชันขนาดใหญ่

ตัวอย่างงาน

- ERP
- Database
- ระบบภายในองค์กร



Container

แพ็กแอปเป็นหน่วยเล็กและพกพาได้

เหมาะกับ

- แอปพลิเคชันสมัยใหม่
- ต้องการความยืดหยุ่น
- พัฒนาและ deploy ได้เร็ว

ตัวอย่างงาน

- Web Application
- Microservices
- API Services



Serverless

รันโค้ดโดยไม่ต้องดูแลเซิร์ฟเวอร์

เหมาะกับ

- งานแบบ Event-driven
- งานที่มีการใช้งานไม่สม่ำเสมอ
- ต้องการลดภาระการดูแลระบบ

ตัวอย่างงาน

- File Processing
- Webhook / API
- ภาพ/วิดีโอประมวลผล
- งานตามเวลา (Scheduled)



Managed Platform

บริการระดับสูงพร้อมใช้งาน

เหมาะกับ

- ต้องการเริ่มใช้งานได้เร็ว
- ไม่ต้องดูแลโครงสร้างพื้นฐาน
- เน้นพัฒนาแอปพลิเคชัน

ตัวอย่างงาน

- Managed Database
- Managed Analytics
- AI/ML Platform
- Application Platform (เช่น PaaS)



"ไม่มีรูปแบบใดดีที่สุดเสมอไป ใช้ให้เหมาะกับงาน จะได้ประสิทธิภาพสูงสุด"

รู้จักกับ Sizing

Sizing คือ การประเมินขนาดทรัพยากรที่ต้องใช้ เพื่อให้ระบบสามารถรองรับงานได้อย่างเหมาะสม
ทั้งในด้านประสิทธิภาพ ความเสถียร และค่าใช้จ่าย



เป้าหมายของ Sizing

- รองรับความต้องการของงาน
- ไม่มากหรือน้อยเกินไป
- คุ้มค่าด้านค่าใช้จ่าย
- รองรับการขยายตัวในอนาคต

> Sizing ที่ดี ไม่ใช่แค่ทำให้ระบบทำงานได้ แต่ทำให้ธุรกิจเติบโตได้อย่างยั่งยืน

ประเภทของ Sizing

การทำ Sizing สามารถพิจารณาได้หลายมิติ ขึ้นอยู่กับลักษณะของงานและเป้าหมายที่ต้องการ



Compute Sizing

ขนาดของหน่วยประมวลผล

ประเมินจำนวน vCPU และหน่วยความจำ (RAM) ที่ต้องใช้

ตัวอย่าง

- 2 vCPU, 8 GB RAM
- 8 vCPU, 32 GB RAM



Storage Sizing

ขนาดของพื้นที่จัดเก็บข้อมูล

ประเมินปริมาณและประเภทของพื้นที่เก็บข้อมูล เช่น SSD, HDD, Object Storage

ตัวอย่าง

- 100 GB (SSD)
- 1 TB (Object Storage)



Network Sizing

ขนาดของการรับส่งข้อมูล

ประเมินปริมาณการใช้งานเครือข่าย เช่น Bandwidth, Data Transfer

ตัวอย่าง

- 100 Mbps
- 1 TB/Month



Scaling Sizing

การเผื่อการขยายตัว

ประเมินการเพิ่มทรัพยากรในอนาคต ทั้งแบบแนวตั้ง (Vertical) และแนวนอน (Horizontal)

ตัวอย่าง

- เพิ่มจาก 2 → 4 vCPU
- Auto Scaling 2–10 instances



"Sizing ที่เหมาะสม ช่วยให้ได้ประสิทธิภาพที่เพียงพอและค่าใช้จ่ายที่คุ้มค่า
พร้อมรองรับการเติบโตในอนาคต"

การคำนวณ Sizing แบบต่าง ๆ

การคำนวณขนาดทรัพยากรช่วยให้เราเลือกสเปกที่เหมาะสม รองรับงานได้เพียงพอ และควบคุมค่าใช้จ่าย

Compute Sizing

คำนวณจำนวน vCPU และหน่วยความจำที่ต้องใช้

สูตรพื้นฐาน

$$\text{Total CPU (vCPU)} = \frac{\text{Peak RPS} \times \text{CPU per request}}{\text{CPU utilization target}}$$

RPS = จำนวนคำขอต่อวินาที (Requests per second)
CPU per request = การใช้ CPU ต่อคำขอ (วินาที)
Target = เป้าการใช้งาน CPU เช่น 70%

ตัวอย่าง

ระบบ Web: 500 RPS, ใช้ CPU 0.02 วินาที/คำขอ เป้า 70%

$$\text{vCPU} = \frac{500 \times 0.02}{0.7} \approx 14.3$$

→ เลือก 16 vCPU

Storage Sizing

คำนวณพื้นที่จัดเก็บข้อมูลที่ต้องใช้

สูตรพื้นฐาน

$$\text{Total Storage} = (\text{Daily Growth} \times \text{Retention}) + \text{Base Data} + \text{Safety Margin}$$

Daily Growth = ปริมาณข้อมูลที่เพิ่มขึ้นต่อวัน
Retention = จำนวนวันที่เก็บข้อมูล
Base Data = ข้อมูลเดิมที่มีอยู่
Safety Margin = เผื่อการใช้งาน (เช่น 20–30%)

ตัวอย่าง

ข้อมูลเพิ่มวันละ 50 GB, เก็บ 365 วัน, ข้อมูลเดิม 200 GB, เผื่อ 20%

Total = (50 × 365) + 200 = 18,450 GB
เผื่อ 20% → 22,140 GB

→ เลือก 25 TB (เช่น Object Storage)

Network Sizing

คำนวณแบนด์วิดที่สำหรับการรับ-ส่งข้อมูล

สูตรพื้นฐาน

$$\text{Bandwidth (Mbps)} = \frac{(\text{Data per request} \times \text{RPS} \times 8)}{1{,}000{,}000} \times (1 + \text{Overhead})$$

Data per request = ขนาดข้อมูลต่อคำขอ (Byte)
RPS = จำนวนคำขอต่อวินาที
Overhead = เผื่อโปรโตคอลและความผันผวน (เช่น 20–30%)

ตัวอย่าง

ขนาดข้อมูลต่อคำขอ 200 KB, 500 RPS, เผื่อ 20%

$$\text{Bandwidth} = \frac{200{,}000 \times 500 \times 8}{1{,}000{,}000} \times 1.2 = 960 \text{ Mbps}$$

→ เลือก 1 Gbps

Scaling Sizing

คำนวณจำนวนอินสแตนซ์สำหรับการขยายตัว

สูตรพื้นฐาน

$$\text{Number of Instances} = \frac{\text{Peak Load}}{\text{Capacity per Instance}}$$

Peak Load
= ปริมาณงานสูงสุด (RPS, Users, etc.)
Capacity per Instance
= ความสามารถของแต่ละอินสแตนซ์ (เช่น RPS ต่ออินสแตนซ์)

ตัวอย่าง

รองรับได้ 300 RPS ต่ออินสแตนซ์, คาดว่า Peak 2,100 RPS

$$\text{Instances} = \frac{2{,}100}{300} = 7$$

→ เลือก 8 อินสแตนซ์ (เผื่อสำรอง)

"การคำนวณ Sizing ช่วยให้ได้ระบบที่เพียงพอ คุ้มค่า และพร้อมรองรับการเติบโต"

ตัวอย่างการคำนวณ Sizing: ระบบเว็บแอป

ตัวอย่างการคำนวณแบบง่าย สำหรับเว็บแอปขนาดเล็ก เพื่อให้เห็นขั้นตอนตั้งแต่โจทย์ → วิธีคำนวณ → ผลลัพธ์

1 โจทย์ *(Requirements)*

ออกแบบระบบเว็บแอปสำหรับองค์กรขนาดเล็ก โดยมีความต้องการดังนี้

ผู้ใช้งานเฉลี่ย
1,000 คน/วัน
(Peak 100 คนพร้อมกัน)

ลักษณะงาน
เว็บแอป + ฐานข้อมูล
(ไม่ซับซ้อน)

ข้อมูลเริ่มต้น
50 GB และเพิ่มขึ้น 10 GB/เดือน

เงื่อนไข
- ต้องการความพร้อมใช้งานที่เหมาะสม
- ควบคุมค่าใช้จ่ายได้

2 วิธีคำนวณ *(Sizing)*

2.1 คำนวณ Compute (Web Server)

ประมาณการใช้ 1 vCPU รองรับได้ ~ 50 concurrent users

จำนวน vCPU ที่ต้องการ = 100 / 50 = **2 vCPU**

เลือกขนาดเครื่อง: **2 vCPU, 4 GB RAM**

2.2 คำนวณ Database

สำหรับงานขนาดเล็ก ใช้ 2 vCPU, 8 GB RAM

เลือกตามแนวทางของผู้ให้บริการสำหรับงานทั่วไป
2 vCPU, 8 GB RAM

2.3 คำนวณ Storage

ข้อมูลเริ่มต้น 50 GB และเพิ่ม 10 GB/เดือน

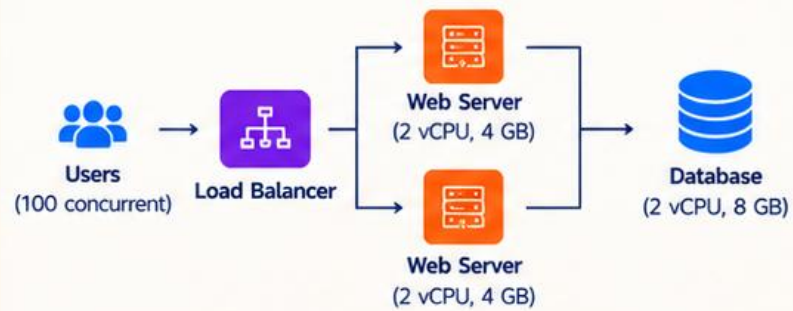
ต้องการใช้งานอย่างน้อย 6 เดือนแรก
50 + (10 × 6) = **110 GB**
เผื่ออีก 20% → 110 × 1.2 = **132 GB**
เลือก: 150 GB (เช่น SSD หรือ Managed Storage)

3 สรุปผล *(Result)*

ขนาดทรัพยากรที่แนะนำ

Web Server	2 vCPU 4 GB RAM (จำนวน 2 เครื่อง เพื่อรองรับความพร้อมใช้งาน)
Database	2 vCPU 8 GB RAM (Managed Database)
Storage	150 GB (รองรับการเติบโต 6 เดือน)

สถาปัตยกรรมโดยรวม (Simplified Architecture)



"การคำนวณ Sizing ที่ดี เริ่มจากความต้องการที่ชัดเจน ใช้หลักการประมาณการที่เหมาะสม และเผื่อการเติบโตในอนาคต"

รูปแบบสถาปัตยกรรมคลาวด์ที่พบบ่อย

เลือกให้เหมาะกับความต้องการด้านขนาด ความซับซ้อน และระดับความพร้อมใช้งานของระบบ

1. Single Server

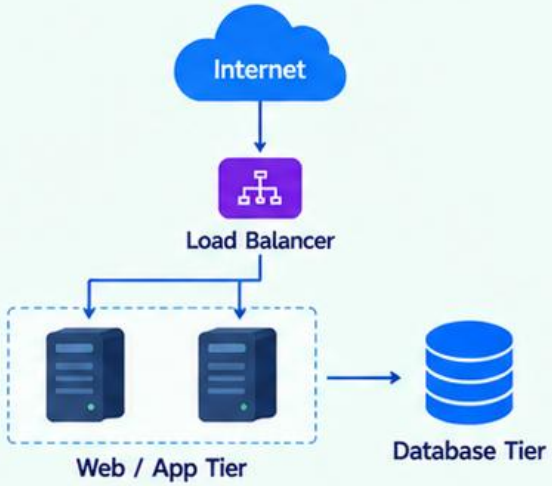
เรียบง่าย เหมาะสำหรับงานขนาดเล็ก



- ✓ โครงสร้างง่าย ต้นทุนต่ำ
- ✓ เหมาะกับงานขนาดเล็ก / ทดสอบ / เริ่มต้น
- ✗ หากเครื่องล่ม ระบบจะหยุดทำงาน

2. Multi-Tier

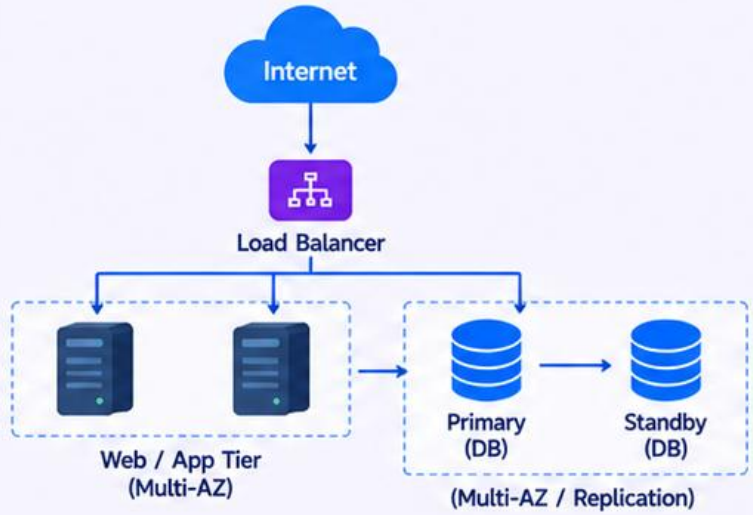
แยกชั้นการทำงาน เพื่อความยืดหยุ่นและปลอดภัย



- ✓ แยกหน้าที่ชัดเจน (Web/App/DB)
- ✓ ขยายแต่ละส่วนได้อิสระ
- ✓ ปลอดภัยและดูแลรักษาง่ายขึ้น
- ✗ ยังไม่มีการสำรองเพื่อความต่อเนื่องสูงสุด

3. Highly Availability (HA)

รองรับการใช้งานต่อเนื่อง แม้เกิดความขัดข้อง



- ✓ มีการสำรองและกระจายข้ามหลาย Availability Zone
- ✓ ลดความเสี่ยงจากความขัดข้อง
- ✓ เหมาะกับระบบที่ต้องการความต่อเนื่องสูง (เช่น ธุรกิจหลัก)
- ✗ ต้นทุนสูงกว่ารูปแบบอื่น

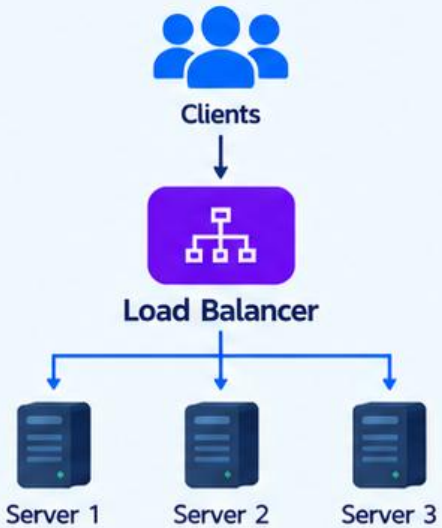
"เลือกรูปแบบสถาปัตยกรรมให้เหมาะกับความสำคัญของระบบ ความเสี่ยงที่ยอมรับได้ และงบประมาณ"

Load Balance และอัลกอริทึมที่ใช้บ่อย

เลือกอัลกอริทึมให้เหมาะกับลักษณะงาน เพื่อเพิ่มประสิทธิภาพ รองรับการใช้งาน และประสบการณ์ที่ดีของผู้ใช้

Load Balancer คืออะไร?

- รับคำขอจากผู้ใช้ (Client)
- กระจายไปยังเซิร์ฟเวอร์หลายเครื่องตามอัลกอริทึมที่กำหนด
- ตรวจสอบสถานะเซิร์ฟเวอร์และไม่ส่งไปยังเครื่องที่ไม่พร้อมใช้งาน



ประโยชน์

- กระจายโหลด ลดการคอขวด
- เพิ่มความพร้อมใช้งาน
- รองรับการขยายตัวได้ง่าย

1 Round Robin

กระจายคำขอไปยังเครื่องถัดไปตามลำดับ



Algorithm (Pseudo-code)

```
index = 0
on each request:
    server = servers[index]
    index = (index + 1) mod N
    return server
```

ข้อดี

- ง่าย ไม่ซับซ้อน
- กระจายโหลดสม่ำเสมอ
- มีค่าใช้จ่ายในการทำงานต่ำ (low overhead)

ข้อจำกัด

- ไม่พิจารณาภาระงานของเซิร์ฟเวอร์
- อาจไม่เหมาะเมื่อแต่ละคำขอใช้เวลาต่างกัน

เหมาะกับ

- งานที่คำขอมีขนาด/ระยะเวลาใกล้เคียงกัน
- เช่น เว็บไซต์ทั่วไป, API ทั่วไป

2 Least Connections

ส่งไปยังเครื่องที่มีจำนวนการเชื่อมต่อน้อยที่สุด



Algorithm (Pseudo-code)

```
on each request:
    server = server with
        minimum active_connections
    return server
```

ข้อดี

- กระจายโหลดตามภาระงานจริง
- เหมาะกับคำขอที่ใช้เวลาต่างกัน
- รองรับความแตกต่างของเซิร์ฟเวอร์ได้ดี

ข้อจำกัด

- ต้องติดตามจำนวนการเชื่อมต่อของทุกเครื่อง
- มีความซับซ้อนมากกว่า Round Robin

เหมาะกับ

- งานที่มีความหลากหลายของคำขอ
- เช่น เว็บแอปพลิเคชัน, ระบบที่มีการประมวลผลใช้เวลาต่างกัน

3 IP Hash

ใช้ค่า IP ของผู้ใช้ในการตัดสินใจ



Algorithm (Pseudo-code)

```
on each request:
    hash_value = hash(client_ip)
    index = hash_value mod N
    server = servers[index]
    return server
```

ข้อดี

- ผู้ใช้เดิมมีกถูกส่งไปเครื่องเดิม (Session Affinity)
- เหมาะกับระบบที่ต้องการเก็บสถานะของผู้ใช้
- ทำงานได้ง่ายและมีประสิทธิภาพ

ข้อจำกัด

- อาจเกิดการกระจายโหลดไม่สมดุล
- เมื่อเพิ่ม/ลดเซิร์ฟเวอร์ การจับคู่ IP อาจเปลี่ยน
- ไม่เหมาะหากผู้ใช้มาจาก IP เดียวกันจำนวนมาก เช่น NAT

เหมาะกับ

- ระบบที่ต้องการคงสถานะผู้ใช้ (เช่น Session)
- เช่น เว็บแอปที่มีการล็อกอิน, อีคอมเมิร์ซ

การขยายระบบ (Scaling) และการทำ Auto Scaling

รองรับการเปลี่ยนแปลงของปริมาณงานได้อย่างยืดหยุ่น อัตโนมัติ และคุ้มค่า

1 เหตุผลในการขยายระบบ

- รองรับจำนวนผู้ใช้งานที่เพิ่มขึ้น
- รักษาประสิทธิภาพและประสบการณ์การใช้งาน
- รองรับการเติบโตของธุรกิจ
- ป้องกันความเสี่ยงจากระบบล่ม
- ใช้ทรัพยากรให้คุ้มค่า ลดต้นทุน

2 ประเภทการขยายระบบ

2.1 Vertical Scaling (Scale Up/Down)

เพิ่มหรือลดขนาดของเครื่องเดิม
เช่น เพิ่ม CPU, RAM, Storage



2 vCPU
4 GB RAM

8 vCPU
16 GB RAM

- **ข้อดี:** ทำได้ง่าย ไม่ต้องแก้สถาปัตยกรรม
- **ข้อจำกัด:** มีขีดจำกัดของเครื่อง (Scale ไม่ได้ไม่สิ้นสุด)
- **เหมาะกับ:** งานขนาดเล็ก–กลาง หรือเริ่มต้นใช้งาน

2.2 Horizontal Scaling (Scale Out/In)

เพิ่มหรือลดจำนวนเครื่องตามปริมาณงาน
เช่น เพิ่ม/ลด VM หรือ Container



2 เครื่อง

4 เครื่อง

- **ข้อดี:** รองรับการเติบโตได้มาก ไม่มีขีดจำกัด และเพิ่มความทนทาน (Fault Tolerance)
- **ข้อจำกัด:** ซับซ้อนกว่า ต้องมี Load Balancer และออกแบบระบบให้รองรับ
- **เหมาะกับ:** ระบบที่มีผู้ใช้งานจำนวนมาก

3 การทำ Auto Scaling

Auto Scaling คือการปรับจำนวนเครื่อง (เช่น VM/Container) โดยอัตโนมัติ ตามเงื่อนไขที่กำหนด
เช่น การใช้งาน CPU, จำนวน Request, หรือเวลา ช่วยให้ระบบมีประสิทธิภาพและประหยัดค่าใช้จ่าย

3.1 องค์ประกอบของ Auto Scaling

Metrics
- CPU uitlization
- Request count
- Memory usage
- Custom metric

→

Auto Scaling Policy
- กำหนดเงื่อนไข
- (Threshold)
- กำหนดจำนวนขั้นต่ำ/สูงสุด
- กำหนดช่วงเวลา (Cooldown)

→

Auto Scaling Group

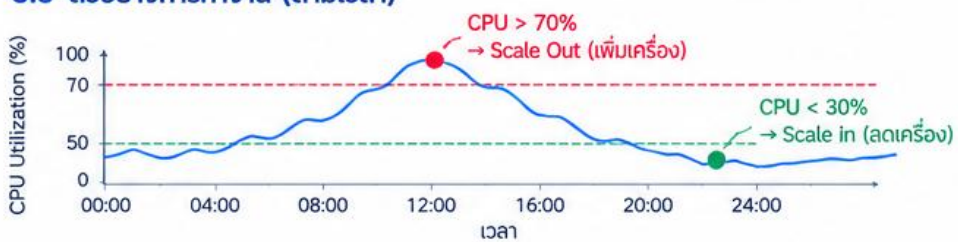


ปรับจำนวนเครื่อง
โดยอัตโนมัติ

3.2 ตัวอย่างเงื่อนไขการทำงาน

เงื่อนไข	การดำเนินการ	ตัวอย่างค่า
CPU > 70% (ต่อเนื่อง 5 นาที)	เพิ่มเครื่อง (Scale Out)	เพิ่ม 1 เครื่อง
CPU < 30% (ต่อเนื่อง 10 นาที)	ลดเครื่อง (Scale In)	ลด 1 เครื่อง
จำนวน Request > 1,000/min	เพิ่มเครื่อง	เพิ่ม 2 เครื่อง

3.3 ตัวอย่างการทำงาน (ตามเวลา)



3.4 ข้อดีและข้อควรพิจารณา

ข้อดี
- รองรับการเปลี่ยนแปลงของโหลดอัตโนมัติ
- ลดความเสี่ยงจากระบบช้า/ล่ม
- ประหยัดค่าใช้จ่าย (จ่ายเฉพาะที่ใช้)
- เพิ่มความพร้อมใช้งานของระบบ
- เหมาะกับงานที่มีความผันผวนของปริมาณงาน

ข้อควรพิจารณา
- ต้องกำหนดเงื่อนไขและทดสอบอย่างเหมาะสม
- มีเวลาในการ Scale (Boot time) อาจไม่ตอบสนองแบบทันทีทันใด
- ควรออกแบบระบบให้ Stateless หรือใช้ Shared Storage
- ระวังการ Scale ขึ้นลงบ่อยเกินไป (Thrashing)
- ต้องมีการ Monitoring และปรับแต่งต่อเนื่อง

การเลือก Cloud Storage

เลือกประเภทพื้นที่จัดเก็บข้อมูลให้เหมาะกับงาน โดยพิจารณาลักษณะงาน ประสิทธิภาพ ความทนทาน และค่าใช้จ่าย

1 ประเภทของ Cloud Storage

Object Storage
เก็บไฟล์/วัตถุจำนวนมาก
เข้าถึงผ่าน API

Block Storage
เหมือนดิสก์สำหรับเครื่องเสมือน (VM)

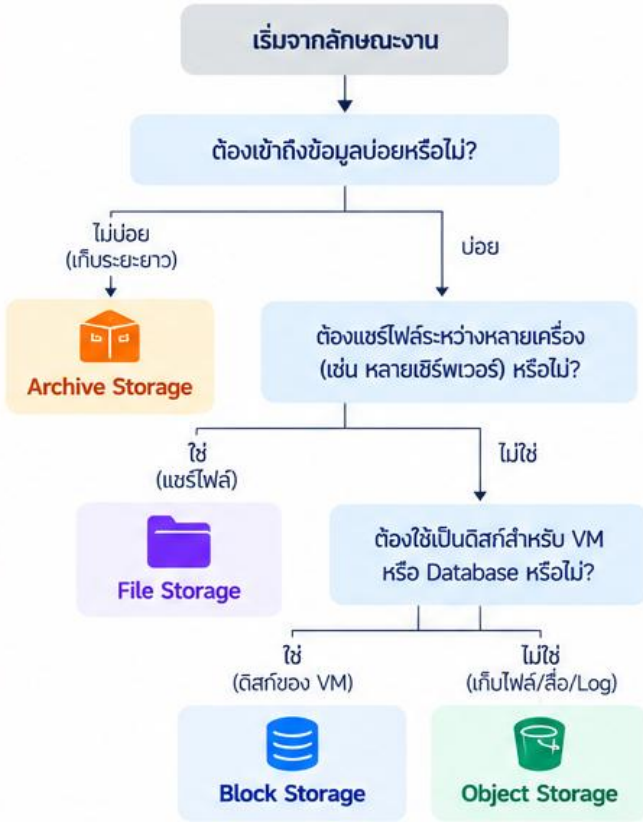
File Storage
แชร์ไฟล์ร่วมกันผ่านโปรโตคอลมาตรฐาน (NFS/SMB)

Archive Storage
เก็บข้อมูลระยะยาว
เข้าถึงไม่บ่อย

2 เปรียบเทียบคุณสมบัติ (โดยสรุป)

คุณสมบัติ	Object	Block	File	Archive
รูปแบบข้อมูล	Unstructured	Block-level	File-level	Unstructured
การเข้าถึง	API/HTTP	Attach to VM	NFS/SMB	API (retrieve)
ความเร็ว (Latency)	ปานกลาง	ต่ำ	ต่ำ–ปานกลาง	สูง (เข้าถึงช้า)
ความทนทาน	สูงมาก	สูง	สูง	สูงมาก
ค่าใช้จ่าย	ต่ำ	ปานกลาง	ปานกลาง	ต่ำที่สุด
เหมาะกับงาน	Backup, Media, Log	VM, Database	Shared files	Long-term retention

3 แนวทางการเลือกใช้งาน



4 ตัวอย่างบริการจากผู้ให้บริการคลาวด์

AWS	Microsoft Azure	Google Cloud
S3 (Object)	Blob Storage (Object)	Cloud Storage (Object)
EBS (Block)	Managed Disk (Block)	Persistent Disk (Block)
EFS (File)	Azure Files (File)	Filestore (File)
S3 Glacier (Archive)	Archive Storage (Archive)	Archive Storage (Archive)

5 ข้อคิดในการเลือก

- พิจารณาความต้องการด้านประสิทธิภาพ ความทนทาน และค่าใช้จ่าย
- เลือกให้เหมาะกับลักษณะงาน (ปัจจุบันและอนาคต)
- คำนึงถึงการบริหารจัดการ ความปลอดภัย และข้อกำหนดด้านกฎหมาย

การออกแบบ Network

ออกแบบเครือข่ายให้ปลอดภัย เชื่อถือได้ และรองรับการขยายตัวของระบบ

1 องค์ประกอบของ Cloud Network

VPC / Virtual Network
เครือข่ายเสมือนของผู้ใช้
แยกจากผู้ใช้อื่น

Subnet
แบ่งเครือข่ายเป็นส่วนย่อย
เช่น Public / Private

Internet Gateway
เชื่อมต่อกับอินเทอร์เน็ต

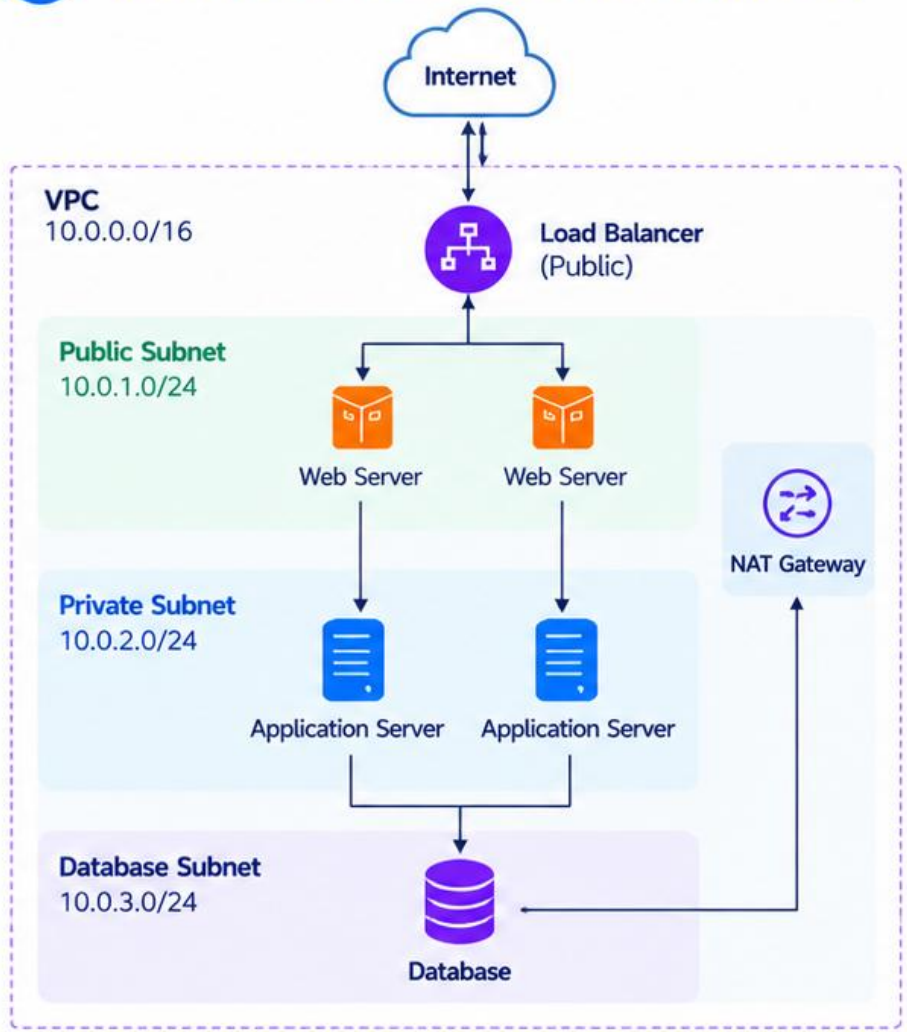
Route Table
กำหนดเส้นทางการส่งทราฟฟิก

Security Group / NACL
ควบคุมการเข้า–ออกของทราฟฟิก
ใช้แบบ Stateful/Stateless

Load Balancer
กระจายทราฟฟิกไปยังหลายเครื่อง

Peering / VPN / Direct Connect
เชื่อมต่อกับเครือข่ายภายนอก
หรือ On-Premises

2 ตัวอย่างสถาปัตยกรรมเครือข่ายบนคลาวด์ (3-Tier)



3 แนวทางการออกแบบ

3.1 แบ่ง Subnet ตามบทบาท
เช่น Public (Web) / Private (App, DB)

3.2 ควบคุมการเข้าถึงด้วย Security Group
อนุญาตเฉพาะที่จำเป็น (Least Privilege)

3.3 ใช้ NAT Gateway
ให้เครื่องใน Private Subnet ออกอินเทอร์เน็ตได้

3.4 ออกแบบเส้นทาง (Route Table)
กำหนดเส้นทางให้เหมาะสมในแต่ละ Subnet

3.5 พิจารณาการเชื่อมต่อกับระบบภายนอก
เช่น VPN หรือ Direct Connect

4 ข้อควรพิจารณา

- ความปลอดภัยของเครือข่ายและการเข้าถึง
- ความพร้อมใช้งาน (High Availability)
- การขยายตัวในอนาคต (Scalability)
- ค่าใช้จ่ายของทราฟฟิก (Data Transfer)
- การปฏิบัติตามข้อกำหนด (Compliance)

ตัวอย่าง: ออกแบบ Cloud Network สำหรับ Web Application

จากโจทย์ความต้องการ ออกแบบเครือข่าย แบ่ง Subnet และกำหนดเส้นทางการเชื่อมต่อ

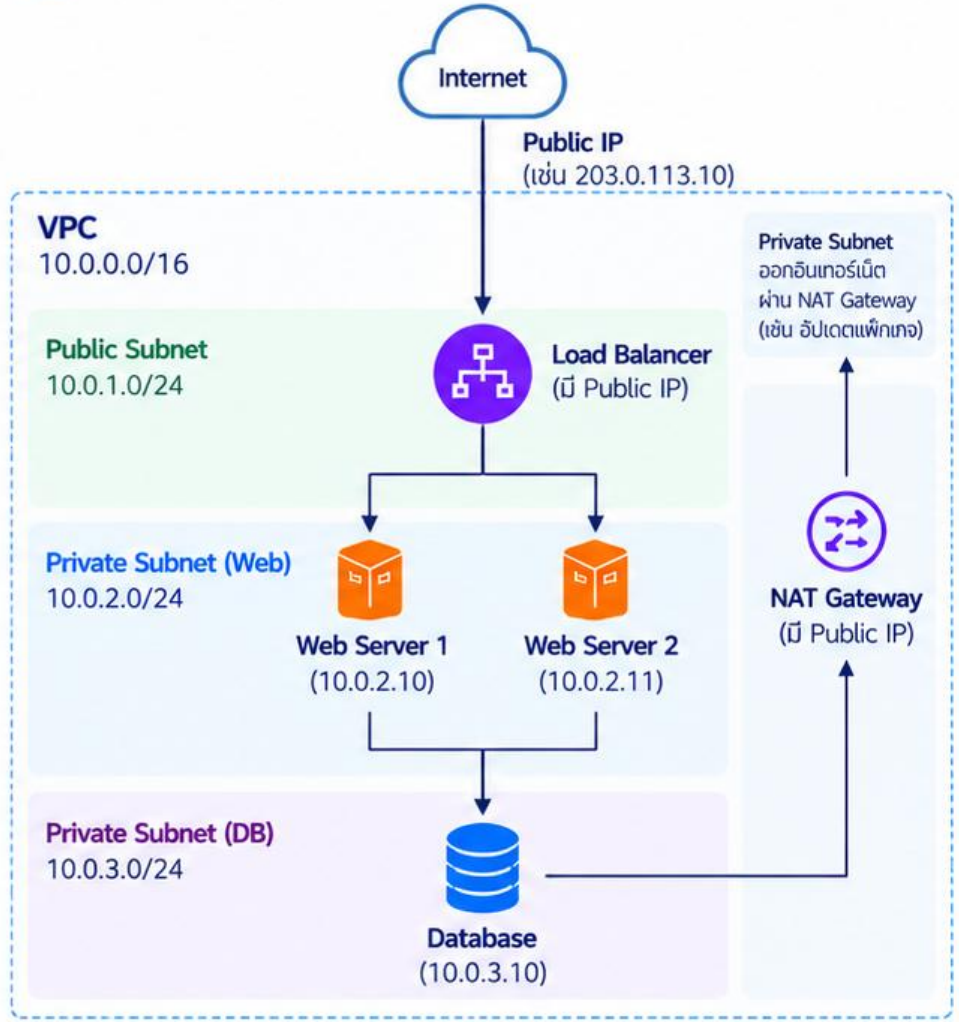
1 โจทย์ความต้องการ

- ให้บริการเว็บไซต์ (Web Application)
- Web Server 2 เครื่อง
- Database 1 เครื่อง
- มี Load Balancer สำหรับรับ traffic จากอินเทอร์เน็ต
- ออกแบบเครือข่ายให้ปลอดภัยและขยายได้

2 แนวทางการออกแบบ

1. สร้าง VPC และแบ่ง Subnet ตามการใช้งาน
2. วาง Load Balancer ไว้ใน Public Subnet
3. วาง Web Server ไว้ใน Private Subnet (ไม่ต้องมี Public IP)
4. วาง Database ไว้ใน Private Subnet (ไม่ต้องมี Public IP)
5. ตั้งค่า Route Table และ NAT Gateway
6. กำหนด Security Group ให้เหมาะสม

3 สถาปัตยกรรมเครือข่าย (Network Architecture)



4 การแบ่ง Subnet และ IP Address

Subnet	CIDR	การใช้งาน	มี Public IP?
VPC	10.0.0.0/16	เครือข่ายหลัก	-
Public Subnet	10.0.1.0/24	Load Balancer	มี
Private Subnet (Web)	10.0.2.0/24	Web Server 2 เครื่อง	ไม่มี
Private Subnet (DB)	10.0.3.0/24	Database 1 เครื่อง	ไม่มี

5 ตัวอย่างการตั้งค่า Security Group (สรุป)

Resource	อนุญาต Inbound	อนุญาต Outbound
Load Balancer	HTTP/HTTPS (0.0.0.0/0)	ไปยัง Web Server (10.0.2.0/24)
Web Server	HTTP จาก LB (10.0.1.0/24)	ไปยัง DB (10.0.3.0/24) และ Internet (ผ่าน NAT)
Database	DB Port (เช่น 3306) จาก Web (10.0.2.0/24)	-

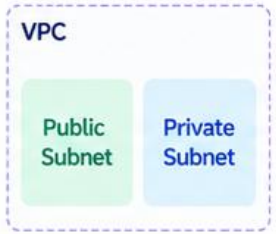
6 สรุปการไหลของ Traffic

1. ผู้ใช้เข้าถึงเว็บไซต์ผ่าน Public IP ของ Load Balancer
2. Load Balancer ส่ง traffic ไปยัง Web Server (Private IP)
3. Web Server เชื่อมต่อกับ Database (Private IP)
4. Web Server ออกอินเทอร์เน็ตผ่าน NAT Gateway (เมื่อจำเป็น)

ศัพท์เครือข่าย (Network Terms) ที่ควรรู้

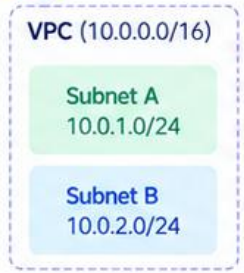
ทำความเข้าใจคำศัพท์สำคัญ เพื่อการออกแบบและใช้งานเครือข่ายบนคลาวด์ได้อย่างถูกต้อง

1 VPC (Virtual Private Cloud)



- เครือข่ายเสมือนบนคลาวด์
- แยกจากผู้ใช้อื่น
- กำหนดช่วง IP, Subnet, Route Table, Security ได้เอง

2 Subnet



- แบ่งช่วง IP ภายใน VPC
- แยกตามหน้าที่/ความปลอดภัย
- มีทั้ง Public และ Private Subnet

3 CIDR (Classless Inter-Domain Routing)



- รูปแบบการระบุช่วง IP (เช่น /16, /24)
- ใช้กำหนดขนาดของเครือข่ายและ Subnet

4 Internet Gateway (IGW)



- เชื่อมต่อ VPC กับอินเทอร์เน็ต
- ทำให้ทรัพยากรใน Public Subnet เข้าถึงอินเทอร์เน็ตได้

5 NAT Gateway



- ให้เครื่องใน Private Subnet ออกอินเทอร์เน็ตได้
- ไม่รับการเชื่อมต่อจากภายนอก
- ใช้สำหรับอัปเดตแพ็กเกจ ดาวน์โหลดไฟล์ ฯลฯ

6 Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	IGW
10.1.0.0/16	Peering / TGW

- กำหนดเส้นทางการส่งทราฟฟิกของ Subnet
- แต่ละ Subnet ผูกกับ Route Table 1 ชุด

7 Security Group (SG)



- Firewall ระดับ Instance (stateful)
- กำหนดกฎอนุญาต/ปฏิเสธตาม Port, IP, Protocol
- ใช้ควบคุมทราฟฟิกเข้า-ออก

8 Network ACL (NACL)



- Firewall ระดับ Subnet (stateless)
- ควบคุมทราฟฟิกเข้า-ออกที่ขอบ Subnet
- มีลำดับกฎ (Rule number) และทั้ง Inbound/Outbound

9 Load Balancer (LB)



- กระจายทราฟฟิกไปยังหลายเครื่อง
- ช่วยเพิ่มความพร้อมใช้งานและรองรับการขยายตัว
- มีหลายประเภท (เช่น ALB, NLB)

10 VPC Peering



- เชื่อมต่อระหว่าง VPC (ไม่ผ่านอินเทอร์เน็ต)
- ใช้สำหรับแชร์ทรัพยากรระหว่าง VPC
- ต้องกำหนด Route ในทั้งสองฝั่ง

11 VPN (Site-to-Site VPN)



- เชื่อมต่อเครือข่ายองค์กรกับคลาวด์อย่างปลอดภัย
- ข้อมูลถูกเข้ารหัส (Encryption)
- เหมาะสำหรับการใช้งานระยะยาว

12 Direct Connect / ExpressRoute



- เชื่อมต่อโดยตรงกับผู้ให้บริการคลาวด์
- ความเร็วสูงและเสถียรกว่าอินเทอร์เน็ต
- เหมาะสำหรับองค์กรที่มีปริมาณข้อมูลมาก

ความปลอดภัยในระบบคลาวด์ (Cloud Security)

ใช้หลายเทคโนโลยีร่วมกัน เพื่อปกป้องระบบข้อมูล และควบคุมการเข้าถึง

1 องค์ประกอบด้านความปลอดภัยที่สำคัญ

Identity & Access Management (IAM)
ควบคุมสิทธิ์การเข้าถึงบัญชีและทรัพยากร

Network Security
ควบคุมการรับ-ส่งข้อมูลในเครือข่าย
(Security Group, NACL, Firewall)

Data Security
ปกป้องข้อมูลทั้งขณะเก็บและขณะส่ง
(Encryption, Key Management)

Application Security
ป้องกันช่องโหว่ของแอปพลิเคชัน
(เช่น WAF, Patch Management)

Monitoring & Logging
ตรวจสอบและบันทึกเหตุการณ์
(CloudWatch, Audit Logs)

Threat Protection
ป้องกันการโจมตีจากภายนอก
(DDoS Protection, WAF, Antivirus)

Compliance & Governance
ปฏิบัติตามนโยบายและมาตรฐานความปลอดภัย
(เช่น ISO 27001, PDPA)

2 Firewall ในระบบคลาวด์

Firewall คือกลไกที่ใช้ “อนุญาต” หรือ “ปฏิเสธ” การรับ-ส่งข้อมูล ตามกฎที่กำหนด
ในระบบคลาวด์มักมี 2 ระดับ

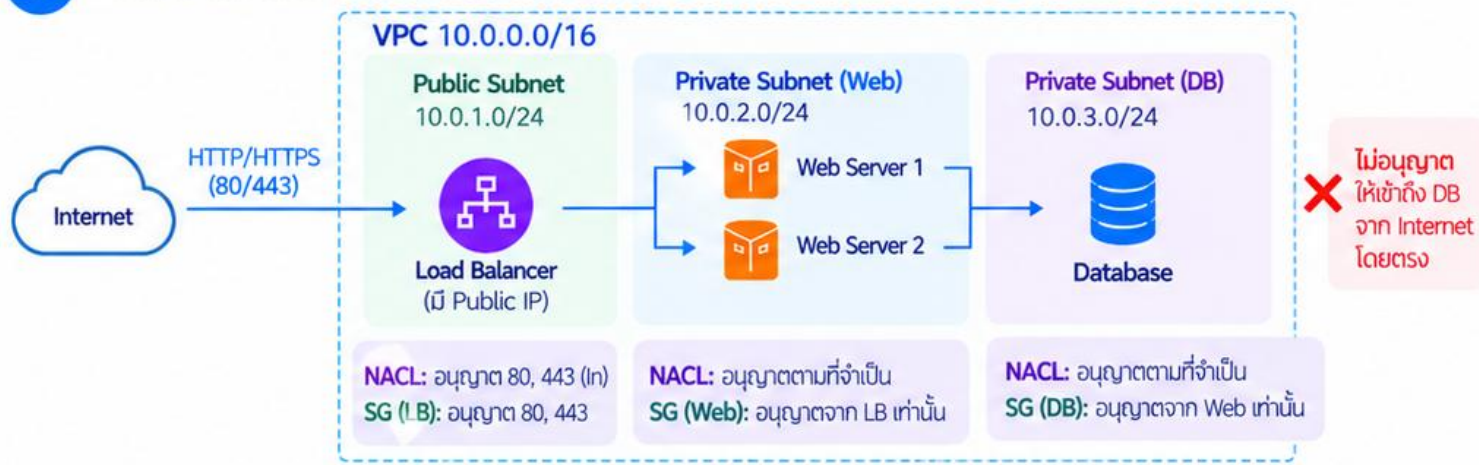
Security Group (SG)

- ระดับเครื่อง (Instance)
- Stateful (อนุญาตขาเข้า จะอนุญาตขาออกกลับให้อัตโนมัติ)
- กำหนดกฎจาก IP, Port, Protocol
- ใช้ควบคุมทราฟฟิกของเครื่อง เช่น Web Server, Database
- มักใช้กับ EC2, Load Balancer

Network ACL (NACL)

- ระดับ Subnet
- Stateless (ต้องกำหนดทั้ง Inbound และ Outbound)
- มีกฎแบบ Allow / Deny และมีลำดับ (Rule number)
- ใช้ควบคุมทราฟฟิกของทุกเครื่องใน Subnet
- เป็นการป้องกันชั้นเพิ่มเติมจาก Security Group

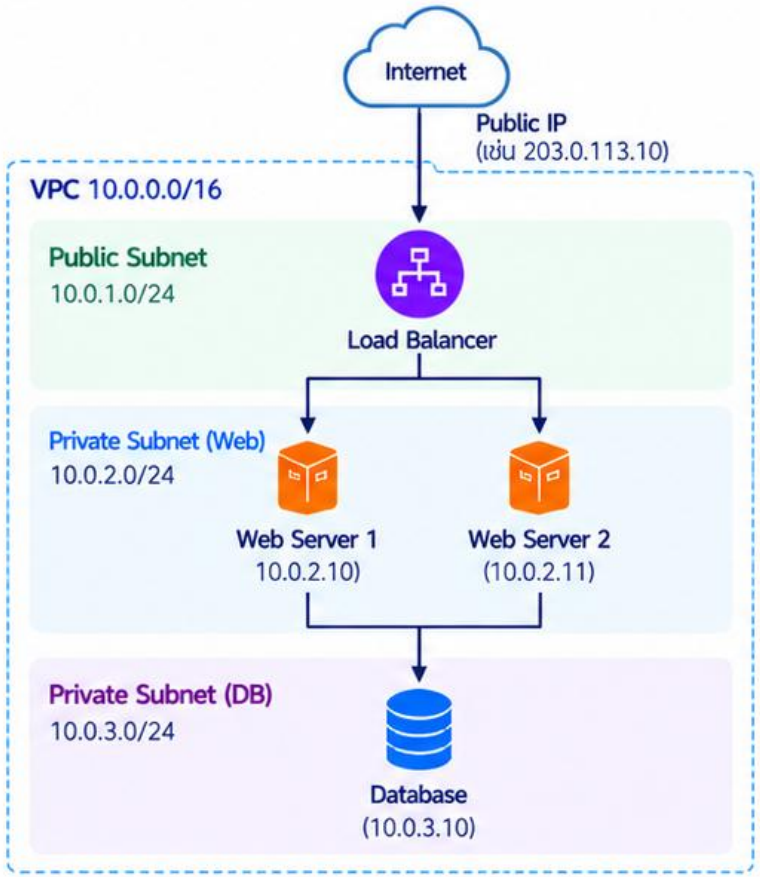
3 ตัวอย่างการใช้ Firewall ในระบบจริง



ตัวอย่างการจัดการ Firewall สำหรับ Web Application

กำหนดกฎ (Rule) ใน Security Group และ Network ACL เพื่อควบคุมการรับ-ส่งข้อมูลให้ปลอดภัย

1 สถาปัตยกรรม (สำหรับอ้างอิง)



2 ตัวอย่างกฎใน Security Group (ระดับ Instance)

Resource	Type	Direction	Protocol / Port	Source	Purpose
Load Balancer	Inbound	TCP 80, 443		0.0.0.0/0	รับ traffic จากอินเทอร์เน็ต
Load Balancer	Outbound	All		10.0.2.0/24	ส่งไปยัง Web Server
Web Server	Inbound	TCP 80		จาก SG ของ Load Balancer	รับ traffic จาก Load Balancer เท่านั้น
Web Server	Outbound	TCP 3306		10.0.3.10/32	เชื่อมต่อ Database
Web Server	Outbound	All		0.0.0.0/0 (ผ่าน NAT)	ออกอินเทอร์เน็ตเพื่ออัปเดตแพ็กเกจ
Database	Inbound	TCP 3306		จาก SG ของ Web Server	รับการเชื่อมต่อจาก Web Server เท่านั้น
Database	Outbound	All		-	ไม่จำเป็นต้องออกอินเทอร์เน็ต

3 ตัวอย่างกฎใน Network ACL (ระดับ Subnet)

Subnet	Rule	Direction	Protocol / Port	Source	Destination	Action
Public (10.0.1.0/24)	100	Inbound	TCP 80, 443	0.0.0.0/0	10.0.1.0/24	Allow
	110	Outbound	All	10.0.1.0/24	0.0.0.0/0	Allow
Private Web (10.0.2.0/24)	100	Inbound	TCP 80	10.0.1.0/24	10.0.2.0/24	Allow
	110	Inbound	All	10.0.0.0/16	10.0.2.0/24	Allow
	120	Outbound	All	10.0.2.0/24	0.0.0.0/0	Allow
Private DB (10.0.3.0/24)	100	Inbound	TCP 3306	10.0.2.0/24	10.0.3.0/24	Allow
	110	Outbound	All	10.0.3.0/24	0.0.0.0/0	Allow

4 ผลลัพธ์ที่ได้

- อินเทอร์เน็ตเข้าถึงได้เฉพาะ Load Balancer (พอร์ต 80, 443)
- Web Server เข้าถึงได้เฉพาะจาก Load Balancer เท่านั้น
- Database เข้าถึงได้เฉพาะจาก Web Server เท่านั้น
- เครื่องใน Private Subnet ออกอินเทอร์เน็ตได้ผ่าน NAT

หมายเหตุ

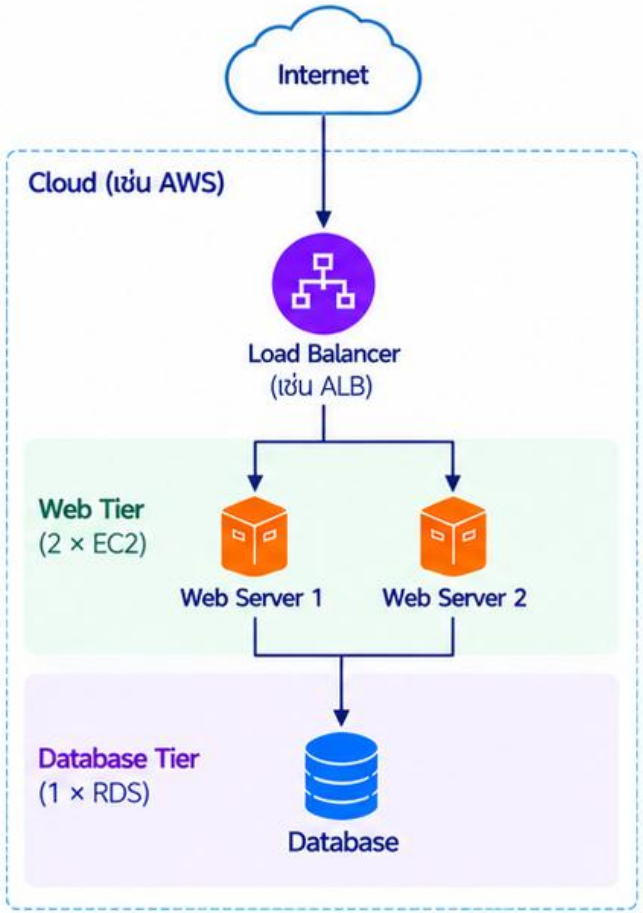
- Security Group เป็น Stateful Firewall (กำหนดเฉพาะ Inbound ก็เพียงพอ)
- Network ACL เป็น Stateless Firewall (ต้องกำหนดทั้ง Inbound และ Outbound)
- ตัวอย่างนี้เป็นเพียงแนวทางพื้นฐาน สามารถปรับให้เหมาะกับนโยบายของ[illegible]ได้

การประมาณการค่าใช้จ่ายและการคำนวณการคืนทุน (Cost Estimation & ROI)

ประเมินค่าใช้จ่ายเพื่อเช่า Cloud เปรียบเทียบกับการลงทุนระบบแบบเดิม และวิเคราะห์ความคุ้มค่าในการย้ายขึ้น Cloud

1 โจทย์และสถาปัตยกรรมที่ออกแบบ

ให้บริการเว็บไซต์ มี Web Server 2 เครื่อง, Database 1 เครื่อง และ Load Balancer ตามที่ออกแบบไว้



2 ประมาณค่าใช้จ่ายบน Cloud (รายเดือน)

รายการ	บริการ (ตัวอย่าง)	จำนวน / สเปก	ราคา/หน่วย (USD/เดือน)	รวมต่อเดือน (USD)	รวมต่อเดือน (ประมาณบาท)
Load Balancer	Application LB	1 ตัว	18.00	18.00	648
Web Server	EC2 (t3.medium)	2 เครื่อง	30.00	60.00	2,160
Database	RDS (db.t3.medium)	1 เครื่อง	52.00	52.00	1,872
Storage	EBS (100 GB/เครื่อง)	3 × 100 GB	10.00	10.00	360
Data Transfer	Internet Outbound	200 GB	18.00	18.00	648
รวมค่าใช้จ่ายต่อเดือน				**158.00**	**5,688**

หมายเหตุ: ใช้ราคาโดยประมาณของ AWS (us-east-1) อัตราแลกเปลี่ยน 1 USD = 36 บาท (ราคาอาจแตกต่างตาม Region และช่วงเวลา)

3 เปรียบเทียบต้นทุนรวม (Total Cost of Ownership) ในระยะ 3 ปี

รายการ	ระบบเดิม (On-Premise)	เช่า Cloud (ตามสถาปัตยกรรมข้างต้น)
ค่าอุปกรณ์/เซิร์ฟเวอร์เริ่มต้น	300,000 บาท	-
ค่าติดตั้งและระบบพื้นฐาน	50,000 บาท	30,000 บาท (Migration)
ค่าไฟฟ้า / พื้นที่ / ดูแลรักษา (8,000 บาท/เดือน)	288,000 บาท (8,000 × 36 เดือน)	-
ค่าเช่า Cloud (5,688 บาท/เดือน)	-	204,768 บาท (5,688 × 36 เดือน)
รวมค่าใช้จ่าย 3 ปี (TCO)	**638,000 บาท**	**234,768 บาท**

ผลลัพธ์: การใช้ Cloud มีค่าใช้จ่ายต่ำกว่าระบบเดิม **403,232 บาท** ในระยะ 3 ปี (ลดลงประมาณ 63%)

4 การคำนวณการคืนทุน (Payback Period)

พิจารณาเงินลงทุนเริ่มต้นในการย้ายขึ้น Cloud (เช่น ค่า Migration) และผลประหยัดค่าใช้จ่ายรายเดือนเทียบกับระบบเดิม

$$\text{Payback Period} = \frac{\text{เงินลงทุนเริ่มต้น}}{\text{เงินประหยัดต่อเดือน}}$$

- เงินลงทุนเริ่มต้น (Migration) = 30,000 บาท
- ค่าใช้จ่ายต่อเดือน (เดิม) = 8,000 บาท
- ค่าใช้จ่ายต่อเดือน (Cloud) = 5,688 บาท
- เงินประหยัดต่อเดือน = 8,000 − 5,688 = 2,312 บาท

$$\text{Payback Period} = \frac{30{,}000}{2{,}312} = 12.97 \text{ เดือน} \cong 13 \text{ เดือน}$$

5 การคำนวณผลตอบแทนจากการลงทุน (ROI)

$$\text{ROI} = \frac{\text{ผลประโยชน์สุทธิ}}{\text{เงินลงทุนเริ่มต้น}} \times 100\%$$

- ผลประโยชน์สุทธิ (Savings 3 ปี) = 403,232 บาท
- เงินลงทุนเริ่มต้น (Migration) = 30,000 บาท

$$\text{ROI} = \frac{403{,}232}{30{,}000} \times 100\% = 1{,}344\%$$

6 ข้อสรุปและข้อพิจารณา

- การเช่า Cloud ช่วยลดค่าใช้จ่ายโดยรวมในระยะยาว
- คืนทุนภายในประมาณ 13 เดือน
- ให้ความยืดหยุ่นในการขยายระบบ (Scale)
- ไม่ต้องลงทุนซื้อฮาร์ดแวร์และดูแลรักษาเอง
- ควรติดตามการใช้งานจริงและใช้เครื่องมือ Cost Monitoring เพื่อลดค่าใช้จ่ายเพิ่มเติม

Assignment: ออกแบบระบบบน Cloud

ออกแบบ สร้างเหตุผล คำนวณขนาด และประมาณค่าใช้จ่าย

ส่งงาน: รายงาน 1–2 หน้า (PDF)
กำหนดส่ง: สัปดาห์หน้า

1 สถานการณ์ (Scenario) (โจทย์)

บริษัทแห่งหนึ่งต้องการพัฒนาระบบขายสินค้าออนไลน์ และต้องการนำระบบขึ้น Cloud โดยมีข้อมูลเบื้องต้นดังนี้

- ผู้ใช้งานประมาณ 30,000 คน/เดือน
- ผู้ใช้งานพร้อมกันปกติ ~ 100 คน ช่วง Promotion อาจเพิ่มเป็น 300 คน
- มี Web/Application Server
- มี Database สำหรับข้อมูลสินค้า ลูกค้า และคำสั่งซื้อ
- รูปภาพสินค้าเริ่มต้นประมาณ 100 GB และเพิ่มขึ้น 20 GB/เดือน
- ระบบต้องเปิดให้บริการ 24 × 7
- ต้องสามารถขยายเมื่อมีผู้ใช้งานเพิ่มขึ้น
- Database ห้ามเปิดให้เข้าถึงจาก Internet โดยตรง
- ต้องการควบคุมค่าใช้จ่าย Cloud

2 งานที่ต้องทำ

2.1 Architecture Design (1 คะแนน)

- ออกแบบ Cloud Architecture ของระบบ
- อย่างน้อยต้องมี Internet → Load Balancer → Web/Application → Database
- สามารถเพิ่มองค์ประกอบอื่นได้ (เช่น Object Storage, Auto Scaling, CDN ฯลฯ)
- อธิบายเหตุผลในการเลือกแต่ละส่วน (สั้น ๆ)

2.2 Sizing & Scaling (1 คะแนน)

- กำหนดขนาดเริ่มต้นของระบบ (เช่น vCPU, RAM, Storage)
- จำนวน Instance (Min / Max)
- กำหนด Auto Scaling Policy อย่างน้อย 1 เงื่อนไข
- อธิบายเหตุผลในการเลือกขนาดและเงื่อนไขดังกล่าว

2.3 Network & Security (1 คะแนน)

- ออกแบบ VPC และ Subnet (ระบุ CIDR)
- กำหนด Public / Private Subnet
- ระบุ Resource ใดมี Public IP / Private IP
- กำหนด Firewall / Security Group ที่จำเป็น
- อธิบายเส้นทางการรับส่งข้อมูล (Traffic Flow)

2.4 Storage Design (1 คะแนน)

- เลือก Storage ที่เหมาะสมสำหรับรูปภาพสินค้า
- เลือก Disk ของ Server และ Database
- กำหนดการสำรองข้อมูล (Backup)
- ระบุประเภท Storage (Object / Block / File / Archive)
- อธิบายเหตุผลในการเลือก

2.5 Cost Estimation (1 คะแนน)

- เลือก Cloud Provider (AWS, Azure หรือ Google Cloud)
- ประมาณค่าใช้จ่ายรายเดือน (Compute, Database, Storage, Network, อื่น ๆ)
- คำนวณค่าใช้จ่ายรายปี (Monthly Cost × 12)
- ระบุสมมติฐานที่ใช้ในการคำนวณ

3 สิ่งที่ต้องส่ง

รายงาน 1–2 หน้า (PDF)

1. Architecture Diagram
2. ตาราง Sizing
3. Network / Subnet Design
4. ตาราง Storage ที่เลือก
5. ตารางประมาณค่าใช้จ่าย (รายเดือน / รายปี)
6. คำอธิบายและเหตุผลในการออกแบบ

4 เกณฑ์การให้คะแนน (5 คะแนน)

หัวข้อ	คะแนน
Architecture Design	1
Sizing & Scaling	1
Network & Security	1
Storage Design	1
Cost Estimation	1
รวม	**5**

5 ข้อกำหนดเพิ่มเติม

- สามารถเลือกบริการและออกแบบแตกต่างกันได้
- ต้องอธิบายเหตุผลในการเลือก
- ใช้ราคาประมาณการจากหน้าเว็บผู้ให้บริการ หรือ Pricing Calculator
- ไม่จำเป็นต้องได้ราคาตรงกันทุกคน
- เน้นความเหมาะสมกับ requirement และการควบคุมค่าใช้จ่าย

6 ตัวอย่างผลลัพธ์ที่คาดหวัง

- ระบบที่รองรับผู้ใช้งานได้ตามที่กำหนด
- ปลอดภัย (Database ไม่เปิดสู่ Internet)
- สามารถขยายได้เมื่อมีผู้ใช้งานเพิ่มขึ้น
- มีการเลือกบริการที่เหมาะสม
- มีการประมาณค่าใช้จ่ายที่สมเหตุสมผล
- อธิบายเหตุผลได้ชัดเจน